1. Ensemble Learning

Ensemble literally means a **group or collection**. In machine learning, **Ensemble Learning** is a supervised technique that combines the predictions of several ML models to build ONE new model with better overall performance than any single ("base") model alone.

The 5 Types of Ensemble Methods

- **Voting (Averaging)**
- **Bagging** (Bootstrap Aggregation) → special case: **Random Forest**
- **Boosting** → AdaBoost, Gradient Boosting, XGBoost
- **Stacking** (a.k.a. Blending / Stacked Generalization)

> *Memory hook: Voting & Stacking use DIFFERENT algorithms on the SAME data. Bagging & Boosting use the SAME algorithm on DIFFERENT/re-weighted data.*

1.1 Voting (Averaging)

- Final prediction = the **average** (for regression) or the **majority / sum** (for classification) of several different models' predictions.
- Base models are usually **different algorithms** (e.g. KNN, Decision Tree, Logistic Regression) trained on the **same dataset**.
- **Hard Voting example:** Data point → sent to KNN, Decision Tree, Logistic Regression → each returns a prediction (1, 0, 1) → the Voting Classifier takes the majority → Final Prediction = 1.

1.2 Bagging (Bootstrap Aggregating)

An ensemble meta-algorithm that improves the **stability and accuracy** of ML algorithms by training the SAME algorithm on many random samples of the data. Done in two steps:

1. **Bootstrapping —** randomly sample the data **with replacement** to create several different training subsets from the original dataset D.
2. **Aggregation —** combine the outputs of all the trained base models (vote / average) into one final, more accurate & lower-variance prediction.

- **Effect:** decreases variance and helps avoid overfitting.
- Usually applied to **Decision Trees**.
- Bagging is a special case of the **"model averaging"** approach.

> *Example from lecture: 1000 students in dataset D → bootstrap samples of 500 students each (D1, D2, D3 …) → a separate SVM is trained on each sample → predictions are combined for a new query point.*

Quick Reference — Bias, Variance, Overfitting, Underfitting

Term	Meaning
Bias	Training error
Variance	Testing error
Overfitting	Training error = LOW, Testing error = HIGH
Underfitting	Training error = HIGH, Testing error = LOW

1.3 Random Forest

- A commonly-used ensemble algorithm — essentially **Bagging applied specifically to Decision Trees**.
- Many decision trees are built (each on its own bootstrapped sample); their individual predictions are then merged/averaged to get one final, more accurate Random Forest prediction.

1.4 Boosting

Builds a **strong classifier from many weak classifiers**, trained **sequentially (in series)** — not in parallel like Bagging.

3. Build a first (weak) model from the training data.
4. Build a second model that specifically tries to **correct the errors** of the first model (misclassified points are given more weight).
5. Repeat, adding models one by one, until either the complete training set is predicted correctly, or the maximum number of models is reached.

- **Popular boosting algorithms:** AdaBoost, Gradient Boosting, XGBoost.

> *Memory hook: Boosting = "sequential error correction" — each new model focuses on fixing the previous model's mistakes.*

1.5 Stacking / Blending / Stacked Generalization

- Stacking, Blending and Stacked Generalization are the **same technique** with different names.
- Uses **heterogeneous (different) weak learners** — e.g. SVM, Logistic Regression, Decision Tree — trained on the same data (like Voting).
- **Key difference from Voting:** instead of a simple vote/average, the outputs of the base models ("Level 0") become the **training data** for a new **Meta-Model** ("Level 1" classifier, e.g. KNN).
- The meta-model itself learns the **best way to combine** the base learners' predictions.
- Combining mechanisms it can learn: majority voting, weighted voting (some models trusted more), averaging, etc.

2. Natural Language Processing (NLP)

NLP is a branch of **AI within Computer Science** that helps computers understand the way humans write and speak. It is difficult because it involves a lot of **unstructured data**, and each person's "tone of voice" is unique and constantly evolving.

Two Components of NLP

A) Natural Language Understanding (NLU)

- Maps natural-language input into **useful (machine-readable) representations**.
- Analyzes different aspects/levels of the language.
- **NLU is HARDER than NLG.**

B) Natural Language Generation (NLG)

Produces meaningful phrases/sentences (natural language) **FROM** an internal representation. Three steps:

6. **Text Planning —** retrieve the relevant content from a knowledge base (Corpus).
7. **Sentence Planning —** choose the required words, form meaningful phrases, set the tone of the sentence.
8. **Text Realization —** map the sentence plan into an actual sentence structure.

Difficulties in NLU — Types of Ambiguity

Ambiguity Type	Description	Example
Lexical	Word-level ambiguity (same word, different meaning/POS)	"google" — noun or verb?
Syntax-level	A sentence can be parsed / structured in more than one way	"Old men and women were taken to a safe place."
Referential	Unclear pronoun reference	"Ali went to Saleem. He said, 'he is tired.'" — who is tired?

Key NLP Terminologies

Term	Meaning	Example
Phonology	Study of organizing sound systematically	—
Morphology	Construction of words from smallest meaningful units (morphemes)	"untestably" → un + test + able + ly
Syntax	Arranging words into grammatically correct sentences; the structural role of words	"Old men and women were taken to a safe place."
Semantics	The meaning of words and how they combine into meaningful phrases	"The car hit the person while it was moving." (ambiguous "it")
Pragmatics	How sentences are understood differently depending on real-world context	"The police are coming." — meaning changes with context
Discourse	How the preceding sentence affects the interpretation of the next one	—

Steps in NLU — the NLP Pipeline

1. Tokenization

- Splitting a phrase / sentence / paragraph / document into smaller units called **tokens** (usually words).
- Example: "Natural Language Processing" → ['Natural', 'Language', 'Processing']
- Needed because we must identify individual words before any further processing — it's the most basic, first NLP step.
- Uses: counting words, counting word frequency, and it's the first step toward stemming/lemmatization.

2. Stemming vs. Lemmatization

Aspect	Stemming	Lemmatization
Method	Chops off last few characters (rule-based)	Uses context + dictionary look-up to find the real base form
Output	Not always a real word — can be erroneous	Always a valid word, called the "Lemma"
Accuracy	Lower	Higher (context-aware)
Speed / cost	Fast, cheap	Computationally expensive (look-up tables + POS tagging)
"Caring" →	"Car" (WRONG)	"Care" (correct)
"Stripes" →	"Strip" (always)	"Strip" (verb) / "Stripe" (noun) — depends on POS
Best for	Large datasets where speed matters	Smaller, homogeneous datasets where accuracy is critical

> *walking / running / swimming → walk / run / swim — stemming and lemmatization give the SAME result here (no ambiguity).*

3. Lexical Analysis

- Identifies and analyzes the structure of words; divides the text into paragraphs → sentences → words, using the **Lexicon** (the collection of words/phrases of a language).
- Example: "The tank was full of water."

4. Syntactic Analysis (Parsing)

- Analyzes the grammar of the words in a sentence and the relationships between them; rejects grammatically invalid sentences.
- Example: "The school goes to boy" → rejected by an English syntactic analyzer.

5. Semantic Analysis

- Extracts the exact / dictionary meaning of the text and checks it for meaningfulness, by mapping syntactic structures onto the task domain.
- Example: "hot ice-cream" → rejected by the semantic analyzer (contradictory).

6. Discourse Integration

- The meaning of a sentence depends on the sentence just before it, and also affects the meaning of the sentence that follows.
- Example: in "He wanted that," the word "that" depends on the prior discourse/context.

7. Pragmatic Analysis

- "Pragmatic" = practical/logical. Deals with how sentences are used and understood in different real situations — contrasted with Semantics (literal/dictionary meaning).
- "Will you crack open the door? I am getting hot." → semantically "crack" means "break," but pragmatically it means "open slightly."
- "If you eat all of that food, it will make you bigger!" → means something different said to a young child vs. an already-obese adult.

Natural Language Generation — Corpus

- A **Corpus** is a very large collection of real text (often billions of words) used to analyze how words/phrases/language are actually used.
- Used to build: predictive keyboards, spell-checkers, grammar correctors, text/speech-understanding systems, text-to-speech modules, machine translation systems, and more.

Types of Corpus

- **Monolingual Corpus —** contains text in ONE language only; usually POS-tagged; used for checking correct word usage, finding natural word combinations, and identifying language trends.
- **Parallel / Multilingual Corpus —** two or more monolingual corpora that are translations of each other (e.g. a novel + its translation); segments must be aligned between the languages so a phrase in one language can be searched and its translation shown.

Real-World Applications of NLP

- Voice-controlled assistants — Siri, Alexa
- Customer-service chatbots (question answering via NLG)
- LinkedIn — scanning listed skills & experience to streamline recruiting
- Grammarly — correcting errors and simplifying complex writing

- Autocomplete / predictive text — trained to predict the next word

How NLP Works

- Structured using different ML methods depending on what's being analysed (e.g. frequency of use, sentiment, or something more complex).
- **NLTK (Natural Language Toolkit) —** a Python library suite for symbolic & statistical NLP in English; supports tokenizing, POS tagging, building text-classification datasets, and more.

3. CNN

Why Not Just Use a Regular ANN?

- A normal ANN fully connects every input neuron to every neuron in the next layer — for images (lots of pixels) this means a huge number of parameters.
- A CNN instead **convolves learned filters/features** with the input using 2-D convolutional layers — far better suited to grid-like 2-D data such as images.
- **Biggest advantage:** CNNs remove the need for MANUAL feature extraction — the network learns the best features by itself.

What Is a CNN?

- A class of neural networks that specializes in processing data with a **grid-like topology**, such as images.
- A digital image is a binary representation of visual data: a grid of pixels, each holding a value that encodes brightness/colour.

The 3 Core Layers of a CNN

1. **Convolutional Layer** — the core building block; carries the main computational load.
2. **Pooling Layer**
3. **Fully Connected Layer**

Overall CNN Pipeline

Input → [Convolution + ReLU → Pooling] (repeated as needed) → Flatten → Fully Connected Layer → Softmax → Output

3.1 Convolutional Layer

- Performs a **dot product** between two matrices: the **Kernel/Filter** (learnable parameters) and a local patch of the input (the "receptive field").
- The kernel is spatially SMALL but matches the full DEPTH of the input — e.g. for an RGB image (3 channels) the kernel is small in height/width but 3 deep.
- **Kernel = a filter** that extracts features; it slides over the input computing a dot product at every position → produces an **Activation Map (Feature Map)**.

3.2 Stride

- The number of pixels the kernel "jumps" by, as it slides across the height and width of the image.
- Larger stride → smaller / coarser output feature map. E.g. a 6×6 image with a 3×3 filter: stride = 1 gives a 4×4 output; stride = 2 gives a smaller 2×2 output.

3.3 Padding

- Adds extra pixels around the border of the image (copied from the edge) so the kernel can properly process edge/corner pixels.
- Fixes the "border effect" problem — without padding, edge pixels get used in far fewer convolution steps than centre pixels.

3.4 Pooling Layer

- Replaces the output at certain locations with a summary statistic of nearby outputs → reduces the spatial size of the representation → less computation and fewer weights needed downstream.
- Applied to every depth slice (every channel / feature map) individually.
- **Max Pooling** — takes the MAXIMUM value in each window (most common).
- **Average Pooling** — takes the AVERAGE value in each window.

> *Example: max-pooling a 4×4 map with a 2×2 filter & stride 2 → keep the largest value from each 2×2 block → 2×2 output.*

Why Pooling?

- Subsampling pixels does NOT change what the object is (a smaller picture of a bird is still recognisable as "bird") → we can shrink the image with far fewer parameters needed to describe it.

A CNN compresses a fully-connected network in **two** ways:

9. Reducing the number of connections (local receptive field, not full connectivity).
10. Sharing the same weights across the whole image (the same filter is reused at every position).

- Max Pooling further reduces the parameter count/complexity on top of this.

3.5 Flattening

- Converts all the resulting 2-D pooled feature maps into ONE long 1-D vector, which becomes the input to the Fully Connected layer.

3.6 Fully Connected (FC) Layer

- Every neuron is connected to every neuron in the previous & next layer — just like a normal ANN.
- Computed as usual: matrix multiplication + bias.
- Its job: map the learned feature representation onto the final output classes (e.g. cat / dog).

3.7 The Filter Intuition — "Beak Detector" Example

- **Insight 1:** some patterns are much SMALLER than the whole image (e.g. a bird's beak) → can be represented with a small region and far fewer parameters.
- **Insight 2:** the SAME pattern can appear in different places (an "upper-left beak" and a "middle beak") → instead of training a separate detector per location, train ONE small detector and let it "move around" (i.e. convolve across the whole image).
- This is exactly what convolution does, and it lets every position share the SAME learned parameters — this is called **weight sharing**.

3.8 Convolution vs. Fully Connected — Why CNN Is Efficient

- A fully-connected layer connects EVERY pixel (e.g. all 36 pixels of a 6×6 image) to every output neuron → huge number of weights.
- A convolution layer connects only a small local patch (e.g. 3×3 = 9 pixels) to each output value, AND reuses the same 9 weights (the filter) everywhere → drastically fewer parameters, and the same "detector" works at every position in the image.

3.9 Colour Images (RGB — 3 Channels)

- A colour image is really 3 stacked 2-D matrices (Red, Green, Blue channels).
- Each filter used on it must ALSO be 3 channels deep (one small matrix per channel) so it can convolve across the full depth of the input at once.

3.10 The Whole CNN — Cat / Dog Example

Image → Convolution → Max Pooling → Convolution → Max Pooling → (can repeat many times) → Flatten → Fully Connected Feedforward Network → Output (cat, dog, …)

3.12 Backpropagation in CNNs

- CNNs are trained the same way as any neural network — using **Backpropagation** (forward pass to compute the output/error, backward pass to update the weights), typically with a sigmoid or similar activation function.
- Typical exercise: given fixed inputs, initial weights, a target output y, and a learning rate, perform one forward pass, then a backward pass (compute gradients & update every weight), then a second forward pass to confirm the error has decreased.

4. Transfer Learning

The Problem With Training Your Own Model From Scratch

1. **Data Hungry** — deep learning models need very large amounts of labelled data.
2. **Time-Consuming** — training is computationally expensive and slow.

What Is Transfer Learning?

- An important deep-learning design technique, especially popular for image/object recognition.

- **Formal definition:** a research problem in ML that focuses on storing knowledge gained while solving ONE problem and applying it to a different but RELATED problem.
- Instead of training a new model from zero, we reuse a model already trained on a large source dataset (e.g. ImageNet) and adapt it to our smaller target task.

Two Key Decisions When Doing Transfer Learning

1. **Which knowledge to transfer —** identify what is common between the source model/task and the target model/task.
2. **Don't degrade performance —** the goal is to IMPROVE the target task, so be careful about where — and when NOT — to transfer knowledge (irrelevant knowledge can cause "negative transfer").

Why It Works — The Feature Hierarchy

CNNs (e.g. VGG-16) learn features in layers, from simple to complex:

- **General / low-level features** (early conv layers) — edges, textures, lines, shapes, colours; task-independent, transfer well to almost any visual task.
- **High-level features** (later/deeper layers):
  - Object parts — eyes, nose, wings…
  - Patterns — e.g. stripes on a zebra
  - Scenes — contextual info: beach, city, forest
  - Whole objects — cars, trees, buildings
- Reference dataset: **ImageNet** (used to pre-train models like VGG-16).

Two Ways of Doing Transfer Learning

1. Feature Extraction

- Use the pre-trained model as a FIXED / frozen feature extractor.
- FREEZE all pre-trained convolutional layers (weights don't update): **conv_base.trainable = False**.
- Add new, untrained classifier layers on top (e.g. Flatten → Dense) and train ONLY those new layers on the target data.
- Fast, needs less data — a good default when the target task is similar to the source task.

2. Fine-Tuning

- Start from the pre-trained weights, then UNFREEZE some of the pre-trained layers too — usually the later/deeper ones, since they hold the more task-specific high-level features.
- Continue training the unfrozen part together with the new classifier layers, using a LOWER learning rate.
- This lets the model adapt its higher-level features to better match the new target task.

> *Keras/VGG16 example: keep blocks 1–4 frozen, unfreeze from "block5_conv1" onward → only the last convolutional block is fine-tuned; earlier, more general blocks stay untouched.*

Summary

Transfer learning lets us reuse knowledge from models already trained on huge datasets to solve new, related tasks with less data and less training time — making it one of the most important and widely-used techniques in modern deep learning, especially for computer vision.

5. Model Selection & Improvement — Cross-Validation

What Is Cross-Validation?

- A statistical technique used to assess the **performance and generalizability** of a predictive model.
- Main purpose: give a MORE RELIABLE estimate of how a model will perform on independent/unseen data than a single train-test split would.

8 Key Reasons Cross-Validation Is Used

1. **Model Evaluation —** evaluates the model on multiple subsets of data for a more robust performance estimate; helps identify over/underfitting.
2. **Reducing Variance —** averaging results across multiple folds reduces variability in the evaluation metric (important with limited data).
3. **Maximizing Data Utilization —** every data point gets used for both training AND testing across the different iterations.
4. **Detecting Overfitting —** a model that performs consistently well across all folds is more likely to generalize well to new data.
5. **Hyperparameter Tuning —** used to evaluate how different hyperparameter choices affect performance.
6. **Model Selection —** used to fairly compare different models/algorithms and pick the best-performing one on average.
7. **Imbalanced Datasets —** ensures every fold has a representative distribution of classes, giving more reliable metrics.

8. **Real-World Performance Estimation —** mimics the real scenario of training on one set of data and testing on a completely different set.

Common Types of Cross-Validation

Type	Description
k-Fold Cross-Validation	Split the dataset into k equal folds; train on (k−1) folds, test on the remaining fold; repeat k times (rotating the test fold) and average the results.
Stratified k-Fold Cross-Validation	Same as k-fold, but each fold preserves the same class-distribution proportions as the full dataset — important for imbalanced datasets.
Leave-One-Out Cross-Validation (LOOCV)	Extreme case of k-fold where k = number of data points (n); each fold uses just ONE data point for testing and all the rest for training.

Summary

Cross-validation is a critical tool for model assessment & selection — it produces more reliable, generalizable performance estimates by making full use of the available data across multiple train/test splits, rather than trusting a single split.